



UNIVERSITY OF PISA

**REPORT FOR IOT PROJECT
IOT APPLICATION FOR MONITORING
COPD PATIENT**

author:

Riccardo Velasco

ACCADEMIC YEAR 2025/2026

1 Introduction

Chronic obstructive pulmonary disease (COPD) is a long-term condition involving inflammation in the lungs, narrowing of the airways and damaged lung tissue, making breathing difficult. Main symptoms of the illness are: breathlessness, chronic cough, fatigue and loss of appetite.

the World Health Organization¹ estimated that the COPD is the third leading cause of death worldwide, causing 3.4 million deaths in 2023; 90% of those were registered in patient aged under 70.

For these reasons, projects like ToLife² were created to leverage artificial intelligence and IoT devices, enabling personalized treatments, better health outcome assessments, and an improved quality of life.

Drawing inspiration from these initiatives, I decided to implement a simple IoT application to monitor the vital signs of hospitalized patients :specifically heart rate, respiratory rate, and SpO2. The system alerts medical staff when a patient enters a critical state and automatically activates an oxygen pump if blood oxygen levels drop below a critical threshold. Additionally, it logs all vital sign data and status changes in a database to enable doctors to review and analyze the patient's medical history.

2 Application Structure

My IoT application consists of a Low Powered and Lossy Network (LLN) and a cloud based application deployed on an unconstrained hospital server. Within the LLN, there are three types of devices:

- **Border Router:** Enables communication between devices within the LLN and external applications and the creation of a DODAG.
- **Sensor Node:** Symulates a smartwatch that tracks vital signs using adaptive sampling governed by an Edge AI model, transmitting the sensed data to the cloud application.
- **Actuator Node:** Comprises an oxygen pump integrated with an alarm system, which receives commands from the cloud application to trigger or deactivate both the alarm and oxygen delivery.

In my implementation all of this devices will be symulated by a dongle nRF52840. The cloud application is divided into two main components:

- **Client:** Receives vital signs from the sensor nodes, analyzes the data to assess patient status, and issues execution commands to the actuator nodes accordingly.

¹World Health Organization. *Chronic obstructive pulmonary disease*. URL: [https://www.who.int/news-room/fact-sheets/detail/chronic-obstructive-pulmonary-disease-\(copd\)](https://www.who.int/news-room/fact-sheets/detail/chronic-obstructive-pulmonary-disease-(copd)).

²Tolife Project. *ombining Artificial Intelligence and smart sensing TOward better management and improved quality of LIFE in chronic obstructive pulmonary disease*. URL: <https://www.tolife-project.eu/project/>.

- **User Interface:** Allows medical personnel to select a patient, view real-time vital signs, review historical patient status, and manually toggle high-frequency adaptive sampling on the sensor node.

3 Implementation

The database selected for this application is **InfluxDB**. Each patient is assigned a specific sensor node and actuator node pair. This coupling, along with the database connection parameters, is stored in a configuration file named `conf.json`, which follows this structure:

```
{
  "applicationName": "COPD ventilator",
  "influxdb": {
    "url": "http://localhost:8086",
    "org": "unipi_iot",
    "bucket": "vital_signs",
    "decision_bucket": "panic_decision"
  },
  "patients": [{
    "patient_name": "Mario Rossi",
    "sensor_nodes": [{
      "sensor_id": 0,
      "ipv6": "fd00::f6ce:367b:6409:f155"
    }],
    "actuator_nodes": [{
      "actuator_id": 0,
      "ipv6": "fd00::f6ce:36d0:9a1d:28c"
    }]
  }]
}
```

3.1 Application protocol

The application layer protocol chosen to connect the LLN devices to the cloud application is CoAP, rather than MQTT. This choice was driven by the following considerations:

- **Design target matches the hardware:** CoAP is explicitly designed for constrained nodes and constrained networks, running over UDP with a compact 4-byte binary header. MQTT requires a persistent TCP connection per client, which implies larger memory buffers, connection handshakes, and keepalive traffic all more costly for the nRF52840 dongles simulating the sensor and actuator nodes.
- **Built-in observation for streaming data:** The CoAP Observe extension lets the cloud client subscribe to a sensor node's vital-signs resource and receive

push notifications whenever new readings are produced, giving MQTT-style publish/subscribe behavior without the need for a broker or a topic.

- **Reliability suited to lossy radio links:** CoAP's Confirmable/Non-confirmable message model provides tunable, per-message reliability appropriate for a wireless multi-hop LLN, where transient link loss is expected. This avoids relying on a long-lived TCP session, which can appear "connected" while silently stalled, requiring additional application-level liveness detection on top of it.

3.2 LLN devices

in this subsection we will analyze in more detail the implementarion of the LLN devices

3.3 Sensor Node

The sensor node simulates a smartwatch equipped with three sensors: Heart Rate (HR), Respiratory Rate (RR), and Oxygen Saturation (SpO2).

3.3.1 Patient simulation

The node will generate data based on three distinct states in wich a simulated patient can be:

- **PATIENT_STABLE:** Simulates a patient in stable condition. Generates data within HR: 60–90 bpm, RR: 12–20 breaths/min, and SpO: 94–99%. The yellow LED remains OFF.
- **PATIENT_CRITICAL:** Simulates a patient requiring medical attention. Generates data within HR: 75–154 bpm, RR: 15–34 breaths/min, and SpO2: 80–95%. The yellow LED is ON (solid).
- **PATIENT_DANGER:** Simulates a patient in immediate danger due to critically low SpO2 levels. Generates data within HR: 135–154 bpm, RR: 32–39 breaths/min, and SpO: 75–84%. The yellow LED is ON (blinking).

to switch between these states you will need to quickly duoble press the button on the dongle.

3.3.2 Data Sampling

The sensor node samples data at a rate that varies depending on its status, which can be:

- **NON_PANIC_MODE:** The patient is considered stable, so data is sampled every 30 seconds to prevent network overload and minimize power consumption. In this state, the dongle LED remains green.

- **PANIC_MODE:** The patient is considered in critical condition, so data is sampled every 5 seconds to minimize delay in cloud-based decision-making. In this state, the dongle LED turns red.

Transitions between these two modes are governed by an Edge AI model (more info in the notebook) that classifies whether the patient is in danger based on measured vital signs. To account for potential misclassifications and address the dangerous nature of a misclassification in this use case, a rule-based override is implemented. Specifically, if $\text{heart_rate} > 130$, $\text{heart_rate} < 40$, $\text{respiration_rate} > 30$, $\text{respiration_rate} < 8$, or $\text{spo2} < 88$, the model is bypassed and PANIC_MODE is immediately triggered.

Additionally, panic mode can be toggled manually by pressing and holding the dongle button for 5 seconds; if the system is already in panic mode, repeating this action reverts it to non-panic mode.

3.3.3 Sending data

The sensor node exposes a CoAP resource named `vital_signs`, which supports Observable GET requests from the client application as well as PUT requests to remotely activate or deactivate Panic Mode (by the client or user).

The `vital_signs` payload sent to the client complies with the SenML standard and is encoded in CBOR to ensure a standardized data representation while minimizing payload size. All the other payload in CoAP messages that are not the vital signs will be simply encoded in json as they are really small.

3.3.4 Handling client unreachability

The sensor node periodically transmits CoAP ping requests to the client to monitor connection status. Standard CoAP retransmission utilizes exponential backoff, which introduces excessive latency in detecting network failures. To overcome this, a custom watchdog timer is implemented to detect communication silence exceeding 15 seconds.

Upon watchdog activation, the sensor node transitions into an offline state and buffers incoming vital sign data in a 5 element circular buffer. Once connectivity is restored, all stored measurements are transmitted to the client.

3.4 Actuator Node

The Actuator Node manages alarm signals and an oxygen pump.

3.4.1 State of the actuator

System states are visually indicated via dongle LEDs as follows:

- **Alarm Status:** When the alarm is inactive, the LED remains solid green. When the alarm triggers, the LED transitions to solid red.

- **Pump Status:** When the pump is powered off, the yellow LED remains solid. When the pump is active, the yellow LED blinks continuously.

Manual control is provided through the dongle button: a single press mutes an active alarm, while a double press deactivates the pump.

3.4.2 CoAP resources

The actuator node exposes two CoAP resources:

- **Alarm:** Allows the client application to trigger or clear the alarm state(via PUT).
- **Pump:** Allows the client application to activate or deactivate the oxygen pump(via PUT).

Additionally, both resources retain a legacy **GET** handler. Although the corresponding remote monitoring feature was discarded in the current iteration, these handlers are preserved to support prospective feature integrations.

3.4.3 Handling client unreachability

To prevent hazardous conditions where a patient remains unmonitored due to system failure, the actuator node implements a 180-second heartbeat monitor. If no communication is received from the client within this interval, the node infers a client outage and automatically triggers the alarm to alert medical staff to intervene.

3.5 Border Router

The border router implementation is the one offered by Contiki in the examples folder.

4 Cloud application

in this section we will discuss the implementation of the two component of the cloud application

4.1 Client

the Client is the main brain of the application, it observe sensor nodes, evaluate health telemetry, actuate responses, and persist data to InfluxDB

4.1.1 System Initialization & Data Hydration

In the configuration phase it loads patient profiles, sensor/actuator IPv6 addresses, and InfluxDB credentials from a `config.json` file. On startup, the client queries InfluxDB for the last 75 seconds of historical data to pre-fill each patient's monitoring window. This prevents the system from needing to build a new baseline from scratch upon restart.

4.1.2 Patient State Evaluation

The client maintains a sliding window of the last 5 readings for each patient and evaluates them against predefined thresholds:

- **Danger Criteria:** $\text{SpO}_2 \leq 85\%$.
- **Critical Criteria:** A reading meets at least two of the following: $\text{HR} \geq 100$, $\text{RR} \geq 24$, $\text{SpO}_2 \leq 92$.
- **Decision Logic:**
 - **Activate Pump:** Triggered if the last 2 consecutive readings indicate Danger.
 - **Activate Alarm:** Triggered if at least 4 readings in the current 5-reading window are Critical.
 - **Stable:** Triggered when all readings in the window are free of Danger and Critical flags.

4.1.3 Automated Control & Actuation

When the patient's evaluated condition changes, the client immediately dispatches CoAP PUT commands. For example, a Danger state sends "1" to the actuator's /pump and /alarm resources, and activates Panic Mode on the sensor. when the patient remains in a "Stable" state, the client explicitly resends stabilization commands every 5 cycles to ensure remote actuators remain synchronized and watchdogs are reset.

4.1.4 Data Persistence

the client decodes CBOR payloads and writes vital signs (`bucket:patient_vitals`), network metrics (`bucket:payload`), and action triggers (`bucket:panic_decisions`) to InfluxDB. If a sensor transmits multiple offline readings in a single payload, the client calculates their correct historical timestamps by iterating backward using the provided interval gaps.

4.1.5 Network Resilience & Heartbeat Server

The client Hosts a local CoAP resource (`/ping`) that responds with an "ACK" to POST requests, serving as the heartbeat destination for the sensor nodes. The client also monitors the CoAP observation stream. If no data arrives within a dynamic timeout period (3 times the last known interval, with a minimum of 10 seconds), the client assumes a network drop, cancels the observation, and initiates a reconnection.

4.2 User

The User application provides an interactive command-line interface for medical staff to manually override sensor states and query historical patient data.

Upon launching the client application, the user is prompted to select a patient. Once selected, four primary actions are available:

- **Activate Panic Mode:** Remotely force the sensor node into PANIC_MODE.
- **Deactivate Panic Mode:** Remotely restore the sensor node to NON_PANIC_MODE.
- **Query Vital Signs:** Retrieve historical vital sign data for the past 15 minutes from InfluxDB.
- **View Event History:** Display the activation history of the decision taken in the last hour.

5 Stress test

To evaluate the robustness of the system's data recovery mechanism, a stress test was designed to simulate a sudden interruption of the connection between the sensor node and the cloud application. The interruption was induced by manually terminating the cloud client process (**Ctrl+C**) while the sensor node continued running normally. To make the effect of a lost connection as visible as possible, the test was run while the sensor node was sampling at its highest rate (5 second interval), so that as many readings as possible would be generated and potentially lost during the outage window. Figure 2 shows the state of the system some time after the disconnection: the readings produced by the sensor node during the outage are missing from the client's view, since the cloud application is offline and cannot receive them. Without a local buffering mechanism on the sensor node, this data would have been lost permanently, as the sensor has no way to deliver it later once no observer is listening.

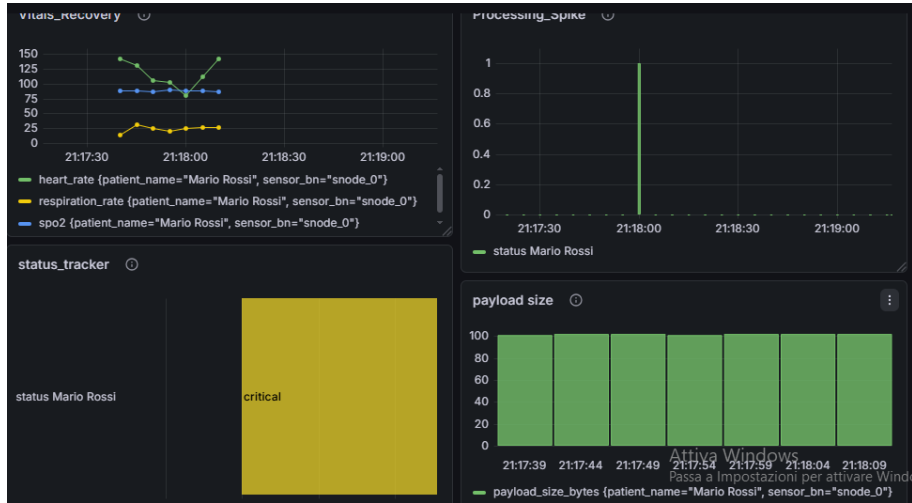


Figura 1: Permanent loss of a packet during the simulated disconnection

figure 2 shows the system shortly after the connection was restored. The readings generated during the outage were not lost: they had been queued in the sensor node's local buffer and were delivered to the client as soon as the observation was

reestablished. This appears as a spike in payload size at the moment of reconnection, since several buffered readings are transmitted together in a single message; CoAP's Block2 transfer mechanism handles this correctly, splitting the larger, multi-reading payload across the wire as needed.

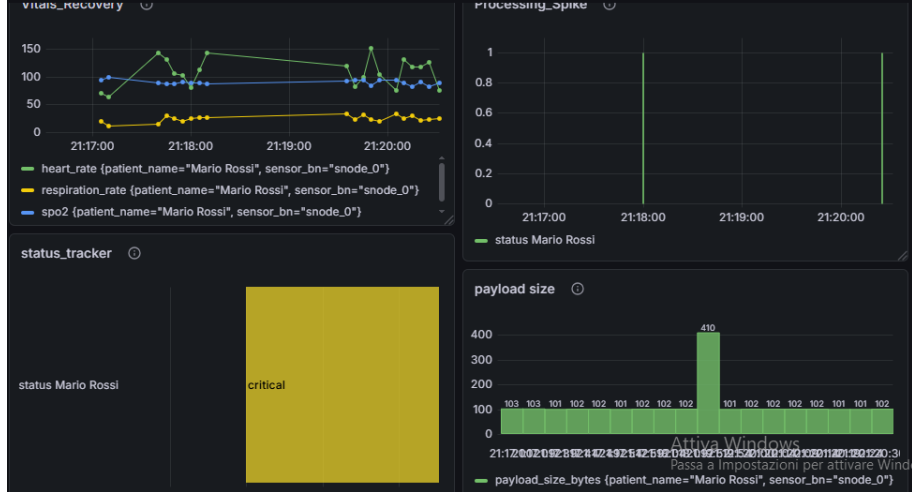


Figura 2: Status after reconnection, showing recovery of buffered data from the sensor node

This test confirms that the sensor node's local buffering strategy prevents data loss during short connectivity interruptions, at the cost of a temporary spike in payload size upon reconnection while the backlog is flushed. Since the buffer has a fixed capacity of 5 readings, this protection is bounded: at the 5-second sampling interval used during panic mode, roughly 25 seconds of outage can be tolerated before the oldest buffered reading is overwritten and permanently lost; at the normal 30-second interval, the same buffer covers a considerably longer window of up to 150 seconds. This limitation is considered acceptable for the intended use case for two reasons. First, the buffer discards only the oldest readings once full, meaning the most recent readings are always preserved and delivered first upon reconnection. Second, patient safety does not rest on data delivery alone: the actuator node enforces its own independent 180 second communication watchdog, which will raise the alarm automatically when detecting radio silence. The buffering mechanism is therefore best understood as protecting the completeness of the historical record for post-incident review.

6 Limitations and Future Work

The current design has a severe limitation that would need to be addressed before a real-world deployment. The architecture has two components whose failure leaves a patient temporarily unmonitored: the Border Router, whose failure disconnects the entire LLN from the cloud application, and the cloud application itself, whose failure or network partition disconnects every patient it manage. This is only partially mitigated by the actuator node's watchdog: in the worst case, a patient could go up

to 180 seconds without any monitoring or corrective action being possible from the cloud side.

A possible improvement would be to let the sensor node, upon detecting that the cloud client is unreachable, communicate directly with the actuator node over the LLN to raise the alarm immediately if its Edge AI model flags a panic condition, rather than waiting for the watchdog timeout. This would meaningfully shorten the worst-case response time during a cloud outage. However, this fallback should be limited to raising the alarm only, and should not be extended to autonomously activating the oxygen pump: administering oxygen is a clinical action with its own risks, and taking it without cloud-side confirmation would mean trusting a lightweight edge model’s classification alone.

7 conclusions

This work resulted in an IoT infrastructure capable of quickly detecting a patient’s deteriorating condition and notifying medical staff of a possible emergency, while autonomously administering oxygen when necessary to bridge the time until medical personnel arrive. A key feature of the system is its adaptive sampling behavior: the sensor node autonomously adjusts its own reporting frequency based on a Edge AI assessment of the patient’s condition, switching to high-frequency sampling as soon as a potential emergency is locally detected, without waiting for an explicit command from the cloud. This same sampling rate can also be adjusted remotely, either automatically by the cloud application, or manually by a user. The sensor node also adapts its behavior when it detects that the cloud client is unreachable: it increases the frequency of its connectivity checks to detect a recovery as soon as possible, and switches to locally buffering incoming readings rather than discarding them, delivering the accumulated backlog automatically once the connection is re-established. The system also maintains a historical record of both the raw vital signs and the history of the decision for each patient.